ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

HPC Lab for CSE

FS 2026
Dr. R. Käppeli,
Dr. F. Gatti, J. Bachmann,
H. Schrader, A. Pietak,
S. Utzinger, V. Vogt,
B. Zemann.

Project 5

Due date: Monday 11 Mai 2026, 18:00 (Europe/Zurich). Please carefully review the information regarding the deadline on the course webpage. Follow the instructions in Section A.

In this project, we will continue learning about parallel programming with MPI which was introduced in project 4. Particularly, we will use the ghost cells exchange between neighboring processes towards building an MPI parallel solver for Fisher's equation that we discussed and parallelized with OpenMP in project 3. Furthermore, we will extend this project with several tasks related to High-Performance Computing with Python. The Python programming language is very popular in scientific computing because of the benefits it offers for fast code development.
As usual, you find all the skeleton source codes for the project on the course GitLab page. We highly recommend to review all the provided skeleton codes before starting any serious implementation of the project tasks.

You may complete this project in groups of up to five students. In fact, we prefer that you do so. Please clearly indicate your collaborators in the "Discussed with" field of your report. Each student must nevertheless prepare and submit their own independent write-up. Refer to Section A for detailed submission instructions.

1 Parallel Space Solution of a nonlinear PDE using MPI [in total 60 points]

This sub-project discusses domain decomposition for an MPI parallel solver of a nonlinear PDE that we discussed in detail in project 3. In project 3, we have added OpenMP to the parallel space solution of a nonlinear PDE mini-application, so that we could use all cores on one compute node on the Euler cluster. The goal of this exercise is now to utilize MPI (Message Passing Interface), enabling the use of multiple compute nodes. Unlike the serial and OpenMP versions, where a single process handles all the data, the MPI version distributes the computational domain across multiple processes (ranks). Each rank handles a sub-domain, allowing for *domain decomposition*. Each process can access only its sub-domain's data and not the data from other processes. For computations using the five-point finite-difference stencil, each process needs data from neighboring grid points. If these points lie on the boundary of a process's sub-domain, the necessary values must be obtained from adjacent processes. Therefore, before each iteration, all MPI processes exchange these boundary values — known as *ghost*, *guard* or *halo cells* — storing them in boundary buffers. This exchange ensures that each process has the necessary data to compute the next iteration.

You can find an initial incomplete version of the MPI code in the directory `mini_app`. The source code is almost equivalent to the serial/OpenMP version that you have already implemented in project 3. There are some comments below and in the code that will guide you through the implementation.

1.1 Initialize/finalize MPI and welcome message [5 Points]

Initialize and finalize the MPI environment in `main.cpp`. Replace the welcome message with one that (i) informs the user that the code is using MPI and (ii) indicates the number of processes:

```
[user@eu-login-39 ]$ salloc --ntasks=4 --constraint=EPYC_7763
salloc: Pending job allocation 56639869
salloc: job 56639869 queued and waiting for resources
salloc: job 56639869 has been allocated resources
salloc: Granted job allocation 56639869
salloc: Waiting for resource configuration
salloc: Nodes eu-a2p-297 are ready for job
[user@eu-login-39 ]$ srun ./main 128 100 0.005
```

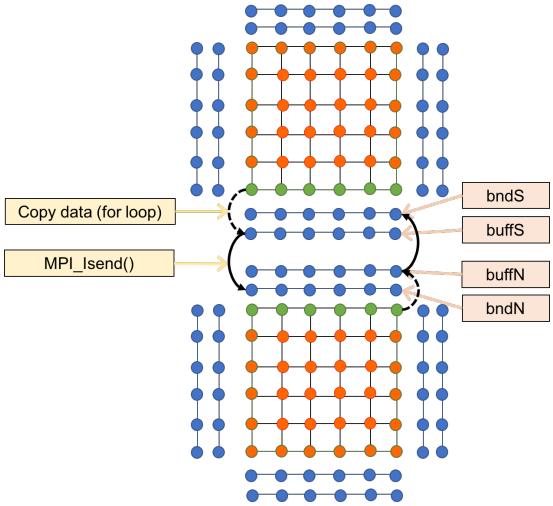


Figure 1: Ghost cell exchange: The bottom process copies the north row (green) into buffer `buffN`, and sends it to its north neighbor (top process). The top process receives the ghost cells from its south neighbor (bottom process) into `bndS`. Likewise, the top process copies the south row (green) into buffer `buffS` and sends it to its south neighbor (bottom process). The bottom process receives the ghost cells from its north neighbor (top process) into `bndN`.

```
================================================================================
                        Welcome to mini-stencil!
version   :: C++ MPI
processes :: 4
mesh      :: 128 * 128 dx = 0.00787402
time      :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
--------------------------------------------------------------------------------
simulation took 0.0592949 seconds
1513 conjugate gradient iterations, at rate of 25516.5 iters/second
300 newton iterations
--------------------------------------------------------------------------------
### 4, 128, 100, 1513, 300, 0.0592949 ###
Goodbye!
```

For readability, only one process should output this message.

1.2 Domain decomposition [10 Points]

Design a domain decomposition strategy that is compatible with any square grid size and any number of MPI processes[1]. Implement this strategy by storing the decomposition details in the `data::Discretization` and `data::SubDomain` structs. In your report, explain your chosen method of domain decomposition. Discuss why this method was selected and analyze its implications on the performance of the application. Consider aspects such as load balancing, communication overhead, and computational efficiency in your analysis.
Hint: Consider the number of data that must be communicated by a generic process with its neighbors. See the short discussion in [1, Sec. 10.4.1].

1.3 Linear algebra kernels [5 Points]

Parallelize the relevant linear algebra kernels `hpc_XXX` in `linalg.cpp` using MPI. Please briefly explain in your report which functions you modified and which you did not.

[1] We naturally exclude the practically irrelevant case where the number of MPI processes exceeds the grid size.

1.4 The diffusion stencil: Ghost cells exchange [10 Points]

Implement the ghost cell exchange between neighboring processes. See Fig. 1 for an illustration. Use *non-blocking* point-to-point communication with the objective to overlap computation and communication. Explain your approach in your report.
Hint: Copy the corresponding cell data into the *send buffers* `buffN`, `buffS`, `buffE` and `buffW`. Similarly, receive the ghost cell data into the *receive buffers* `bndN`, `bndS`, `bndE` and `bndW`.

1.5 Implement parallel I/O [10 Points]

Implement the output of the computed solution with MPI-I/O.
Hint: Study the MPI-I/O demo in the provided skeleton codes.

1.6 Strong scaling [10 Points]

How does your code scale for different resolutions? Plot the time to solution for $N_{\text{CPU}} = 1, 2, 4, 8, 16$ processes across resolutions of $n \times n$, where $n = 64, 128, 256, 512, 1024$. Interpret your results and compare them to the OpenMP implementation of Project 3 in your report.

1.7 Weak scaling [10 Points]

How does code scale for constant work by process ratio? Plot the time to solution as the total problem size and the number of processes $N_{\text{CPU}} = 1, 4, 16, 64$ increase proportionally, to maintain a constant workload per process, and the base resolutions $n \times n$ and $n = 64, 128, 256$. Interpret your results and compare them to the OpenMP implementation of Project 3 in your report.

1.8 Bonus [20 Points]: Overlapping communication/computation details

Verify if the used MPI implementation is truly overlapping computation with communication (i.e., confirm that the non-blocking communication is truly asynchronous). If it is not, can you modify your implementation to achieve a true overlap? How does the parallel scaling of your code change as a result?
Hint: See the discussion in [1, Sec. 10.4.3].

2 Python for High-Performance Computing [in total 40 points]

Python is increasingly used in High-Performance Computing (HPC) projects, serving various roles such as a high-level interface to existing applications and libraries, an embedded interpreter, or even for direct implementation. Its popularity in scientific computing has surged due to its flexibility and ease of use. Users now commonly use Python not only to prototype codes at small scales but also to develop parallel production codes. This shift is partly replacing traditional compiled HPC languages such as C/C++ and Fortran for certain applications. However, when adopting Python for such purposes, it is crucial to monitor performance to meet the rigorous demands of HPC environments. Similar bindings exist for other popular languages such Julia[2] (see MPI.jl[3]).

We highly recommend to (partly) watch the course High-Performance Computing with Python[4] held July 02–04, 2019 at CSCS. In particular, we will use the package MPI for Python (`mpi4py`) for using MPI within Python. To get started, please watch the Introduction to MPI[5] lesson of the CSCS course. Although the lessons use mostly IPython/Jupyter notebooks, we will use plain Python scripts.

Cluster environment setup instructions

For this project, we will need to install several Python packages. We recommend using a virtual environment for this purpose:

```
[user@eu-login]$ module load stack gcc openmpi python # loads necessary modules
[user@eu-login]$ python -m venv project05-env # creates a virtual environment
[user@eu-login]$ source project05-env/bin/activate # activates the virtual environment
(project05-env) [user@eu-login]$ pip install numpy scipy matplotlib mpi4py # installs
↪  the necessary packages (this may take some time)
```

[2] https://julialang.org/
[3] https://juliaparallel.org/MPI.jl/stable/
[4] https://www.youtube.com/watch?v=JYX4TQ_fCqY&list=PL1tk5lGm7zvQ-EzsiTZ6Xv1SxZs74epzg
[5] https://www.youtube.com/watch?v=XeyspDaKjMM

Please note that you will have to load the above modules and activate the virtual environment for the rest of this task. We refer to the documentation for any further information on the management of virtual environments in Python.[6]

For Python, we refer to the documentation

- https://docs.python.org/3/

The documentation for `mpi4py` can be found here

- https://mpi4py.readthedocs.io/en/stable/index.html

Remember to use the help function within a Python interpreter:

```
>>> from mpi4py import MPI
>>> help(MPI)
```

In order to get started, we begin with a simple Python MPI program `hello.py`:

```
from mpi4py import MPI

# get comm, size, rank & host name
comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
proc = MPI.Get_processor_name()

# hello
print(f"Hello world from processor {proc}, rank {rank} out of {size} processes")
```

Run the script as follows:

```
(project05-env) [user@eu-login]$ salloc -n 8 --nodes=4 --ntasks-per-node=2 # allocate
↪  8 processes on 4 nodes with 2 processes each
(project05-env) [user@eu-login]$ srun python3 hello.py
Hello world from processor eu-a2p-477, rank 0 out of 8 processes
Hello world from processor eu-a2p-477, rank 1 out of 8 processes
Hello world from processor eu-a2p-480, rank 4 out of 8 processes
Hello world from processor eu-a2p-480, rank 5 out of 8 processes
Hello world from processor eu-a2p-481, rank 7 out of 8 processes
Hello world from processor eu-a2p-481, rank 6 out of 8 processes
Hello world from processor eu-a2p-478, rank 2 out of 8 processes
Hello world from processor eu-a2p-478, rank 3 out of 8 processes
```

Now that everything is set up and working, we can get started!

2.1 Sum of ranks: MPI collectives [5 Points]

With MPI for Python's collective communication methods, write a script that computes the sum of all ranks:

- using the pickle-based communication of generic Python objects, i.e. the *all-lowercase* methods;
- using the fast, near C-speed, direct array data communication of buffer-provider objects, i.e., the method names starting with an *uppercase* letter.

2.2 Ghost cell exchange between neighboring processes [5 Points]

Write a script that creates a two-dimensional, periodic Cartesian topology and implements ghost cell exchange (as in Project 4):

- use the method `MPI.Compute_dims`, a convenience function similar to MPI's `MPI_Dims_create`;
- create a Cartesian topology using MPI for Python;
- determine the neighboring processes;

[6] See here https://docs.python.org/3/library/venv.html.

- output the topology: rank, Cartesian coordinates in decomposition, East/West/North/South neighbors;
- for each process, exchange the ghost cells with the four east/west/north/south neighbors.

Verify that you obtain the expected result.

2.3 A self-scheduling example: Parallel Mandelbrot [30 Points]

In this task, you are asked to implement one of the most common parallel patterns: the *manager-worker* pattern. The basic idea is that one process, known as the manager, is responsible for delegating work to other processes, known as the workers. This is particularly useful in problems where the amount of work per worker is difficult to estimate and the workers don't have to communicate with each other in order to do their work. As a particular example, we again consider the Mandelbrot set. Note that this is only meant as an illustration of this fundamental type of parallel algorithm, and not really as the best way to parallelize the computation of the Mandelbrot set.

The manager decomposes the Mandelbrot set into a number of (rectangular) patches. Computing the Mandelbrot (sub)set on a particular patch will be called a task. The manager then delegates these tasks to the workers. Once a worker is done computing a particular task, he sends the patch back to the manager. Therewith, the worker signals to the manager that he is available to work on a new task. The manager then sends the worker another task to work on. This process is repeated until no more tasks remain, i.e., all the patches of the Mandelbrot set have been computed. Finally, the manager combines all the patches from the workers and outputs the Mandelbrot set.

The skeleton codes for this sub-project are located in the folder `hpc_python/ManagerWorker` available through the course GitLab page. Begin by familiarizing yourself with the `mandelbrot_task.py` module. It contains two classes. First, the class `mandelbrot`, which decomposes the Mandelbrot set computation in a series of subsets or patches, produces a list of tasks, and combines the tasks' patches together. Second, the `mandelbrot_patch` class, which holds a subset or patch of the Mandelbrot set and contains a method `do_work` that performs the actual computation. This part is already fully implemented for your convenience. However, feel free to try out different implementations, e.g., domain decompositions, etc. Complete the following:

- Implement the manager-worker algorithm in the skeleton code `manager_worker.py`.
- Add a scaling study using 2-32 workers for a `4001x4001` domain and split the workload into 50 and 100 tasks.

The program can be called as follows:

```
(project05-env) [user@eu-login]$ salloc --ntasks=4 --constraint=EPYC_7763
(project05-env) [user@eu-login]$ srun python3 manager_worker.py 4001 4001 100
```

A Additional notes and submission details

Submit your **report** and **all source code files**, including your build files (e.g., `Makefile`(s)) and other scripts (e.g., batch job scripts, plotting scripts), as a compressed tar archive. Your LaTeX report must be based on the provided template from the course webpage and should summarize your results and observations for all sections. Your submission should be uploaded to SAM-UP and must include

- Your report, named as `project_number_lastname_firstname.pdf`. You may discuss all questions with anyone you like. However, (i) you must list all collaborators in your submission, and (ii) you must prepare and write up the submission independently.
- A compressed tar[7] archive, named `project_number_lastname_firstname.tar.gz`, containing:
  - All source code files.
  - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
  - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
- The total size of your submission (report PDF and archive) must not exceed 25 MB.

[7] https://en.wikipedia.org/wiki/Tar_(computing)

- File names must contain only `ASCII` characters (avoid spaces, special symbols, and non-English letters).

Please strictly follow these instructions and naming conventions. Failure to do so results in additional work for the teaching assistants.

References

[1] Georg Hager and Gerhard Wellein. Introduction to high performance computing for scientists and engineers. *Chapman & Hall/CRC Computational Science*, July 2010. URL: http://dx.doi.org/10.1201/EBK1439811924, doi:10.1201/ebk1439811924.